Cheaper Re-planning, not Less Re-planning: An Efficiency Study of SmolVLA on a Turing GPU

Minh-Ngoc Luong
Hanoi University of Science and Technology
ngoclm004@gmail.com

Abstract

A vision-language-action (VLA) policy that re-plans at every control step pays for the whole model at every step. This report measures where that cost goes for the released LIBERO checkpoint of SmolVLA on a Tesla T4, and tests which ways of reducing it preserve closed-loop success. A re-plan takes 789 ms, dominated by emulated bfloat16 arithmetic and a launch-bound 10-step flow loop. On identical episodes with paired tests, re-planning less often (open-loop chunks, gripper-event triggers, a 5-step-old image) lowers success by 12.0–12.5 points (Holm $p \leq 0.012$), whereas re-planning more cheaply (float16, 3 Euler steps, a bit-exact CUDA-graph replay) cuts a re-plan to 112 ms (7.0×) with unchanged success on 200 episodes (77.0% vs 77.0%). ***[500-pair non-inferiority result.]*** *Code: https://github.com/Ngoc-LM/efficient-vla.*

1. Introduction

VLA policies map images, a language instruction and the robot state to actions through a pretrained vision-language model (VLM) and an action head [1, 17]. At batch size one inside a control loop, every re-plan waits for the full model. Efficient-VLA work reduces this cost with smaller backbones, fewer tokens or layers, cheaper action decoding, asynchronous execution or faster inference systems (Sec. 2), but a saving can change closed-loop behaviour: an ONNX export of the SmolVLA checkpoint used here cut median latency by 40% and, through truncated instructions, LIBERO-Spatial success from 70% to 41% [6].

This report studies SmolVLA [17] on LIBERO-Spatial [12] with a Tesla T4, a GPU without bfloat16 tensor cores, in a training-free setting: weights, architecture and protocol stay fixed, and only the execution changes. It surveys efficient VLAs and diffusion versus flow-matching action heads (Sec. 2, with an 18-paper literature matrix), reproduces the released checkpoint (Sec. 3), analyses its strengths, limitations and bottlenecks (Sec. 4), and tests two hypotheses on identical episodes: re-planning less often, and re-planning more cheaply (Sec. 5). The levers that make re-planning cheaper are known [13, 19]; the contribution is a measured account of why each helps on this hardware and a paired closed-loop test of whether success is preserved.

Lever	Examples	Reported effect (hardware)
Smaller model	SmolVLA [17], TinyVLA [20]	0.45B, LIBERO avg 87.3; 14 vs 292 ms per action vs OpenVLA (A6000)
Fewer tokens or layers	VLA-Cache [21], EfficientVLA [22], DeeR [23]	1.63× (4090); 1.93× at 28.9% FLOPs, −0.6 SR (A40); LLM FLOPs 5.2–6.5× lower (trained)
Action decoding	FAST [15], OFT [7], CogACT [10]	≤ 5× faster training, 750 vs 100 ms per chunk; 26× throughput; VLM once, small DiT denoises
Execution	RTC [2], A2C2 [16]	>300 ms delay tolerated, +21 ms per chunk; 4.7 ms trained head per step
Systems	Realtime-VLA [13], ONNX [6], Coda [19]	π_0 106.5→27.3 ms (4090); −40% latency, SR drop; 2 flow steps + learned corrector

Table 1. Efficiency levers (selection of the literature matrix), as reported by each paper.

2. Efficient VLAs and Action Generation

Efficiency levers. Tab. 1 groups the efficiency work in the literature matrix by the part of the inference loop it changes. Token and layer reduction assumes redundancy across frames [21] or easy situations [23] and saves compute only in the language model. Chunked and asynchronous execution assumes that actions predicted from an older observation stay valid; SmolVLA's asynchronous mode finished tasks 30% faster but lowered success from 78.3% to 73.3% [17]. Systems work assumes the model stays numerically the same, which an export can violate [6]. Two questions remain open: speed-ups are reported next to aggregate success rates, although a non-significant difference is not equivalence [6], and most latencies come from GPUs with native bfloat16 (RTX 4090, A100, A40).

Diffusion versus flow matching. Both turn Gaussian noise $\boldsymbol{\epsilon}$ into an action chunk $\mathbf{A}$ conditioned on the observation $\mathbf{o}$. Diffusion heads [3, 10] noise the data as $\mathbf{A}_k = \sqrt{\bar{\alpha}_k}\mathbf{A} + \sqrt{1-\bar{\alpha}_k}\boldsymbol{\epsilon}$ and predict the noise [4]; flow-matching heads [1,

17] interpolate linearly, $\mathbf{A}^\tau = \tau\boldsymbol{\epsilon} + (1-\tau)\mathbf{A}$, and regress the velocity of that path [11]:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{A}_k, k \mid \mathbf{o})\right\|^2, \quad (1)$$

$$\mathcal{L}_{\text{FM}} = \mathbb{E}\left\|(\boldsymbol{\epsilon} - \mathbf{A}) - v_\theta(\mathbf{A}^\tau, \tau \mid \mathbf{o})\right\|^2. \quad (2)$$

They differ in four respects. (i) Target: noise versus velocity. (ii) Path: the linear path gives each training pair a constant velocity and yields straighter sampling trajectories [11]; on RoboTwin, $\pi_{0.5}$ keeps 71.3% success with 3 flow steps against 71.5% with 10, and drops to 59.8% with one [19]. (iii) Sampler: Diffusion Policy uses 100 DDPM steps in simulation and 10 DDIM [18] steps on the robot, CogACT 10 DDIM steps with guidance, flow models 10 Euler steps. (iv) Architecture: Diffusion Policy repeats its whole denoiser; CogACT, π_0 and SmolVLA run the VLM once and repeat only a head on its cached output, and for SmolVLA such a step is dominated by kernel launches at batch one (Sec. 4).

3. Reproduction

Model and setup. The study runs the released LeRobot implementation of SmolVLA (v0.6.1) with the checkpoint `HuggingFaceVLA/smolvla_libero` on Kaggle (2× Tesla T4, PyTorch 2.11, MuJoCo 3.3.7, EGL rendering). The checkpoint differs from the paper configuration [17]: it keeps all 32 decoder layers of SmolVLM2-500M instead of 16, uses an action expert of width $0.5\times$ instead of $0.75\times$, and executes one action per predicted chunk (n_{exec}=1). It has 605M parameters (vision encoder 86M, connector 12M, language model 362M, action expert 97M) stored in bfloat16.

Pipeline. At control step k the policy receives two RGB images (third-person and wrist, resized with padding to 512^2), the instruction (at most 48 tokens) and an 8-D state (end-effector pose, two gripper joints). SigLIP encodes each image, a pixel-shuffle connector shortens it to a few tokens, and the language model processes the prefix [image; language; state] once, caching its keys and values per layer. The action expert, whose layers alternate cross-attention to this cache and self-attention over action tokens, predicts a chunk $\mathbf{A} \in \mathbb{R}^{50\times 7}$ (end-effector delta pose and gripper). It is trained with $\mathcal{L}_{\text{FM}}$ (Eq. (2)), with $\tau \sim \text{Beta}(1.5, 1)$ rescaled to $[0.001, 1]$. At inference, S=10 Euler steps $\mathbf{A} \leftarrow \mathbf{A} - \frac{1}{S} v_\theta(\mathbf{A}, \tau \mid \mathbf{o})$ integrate from τ=1 (noise) to 0; a re-plan thus costs one prefix pass and S expert passes. The first action is unnormalised and executed at 20 Hz.

Metric and protocol. Success rate (SR) is the fraction of episodes whose goal predicate (*e.g.* `On(bowl, plate)`) holds within 280 steps. An episode is a (task, stored initial state) pair; the 10 tasks run on initial states 0–19 (200 episodes; 0–49 for the main comparison). Methods are compared on identical episodes with common random numbers: the flow noise at step k is seeded by (episode, k) for every method.

Component (one re-plan)	GFLOP	bf16 (ms)	fp16 (ms)
Vision encoder, 2 cameras	431	230	39
Prefix incl. vision (32 LM layers)	527	304	82
One expert pass (flow step)	11.5	51	52
Re-plan, S=10 (released)	642	814	605
Simulator step (CPU + EGL)	–	32	

Table 2. Cost of one re-plan on a Tesla T4 (isolated components, one profiling run; end-to-end times in Tab. 3 come from a separate run and agree within 3%).

Differences and result. MuJoCo is pinned below 3.4, from which the stored initial states of task 5 settle differently and SmolVLA drops from 80% to 28% on that task [9]; the evaluation uses 20 instead of 10 episodes per task. The reproduced SR of 77.0% [70.7, 82.3] (Wilson 95% CI) is below the 90% reported for the paper configuration but inside the 70–83% obtained by others with this checkpoint [6, 8].

4. Analysis

Computational bottlenecks. Tab. 2 breaks down one re-plan (CUDA events, warm-up, median of 60 calls). The simulator is not the bottleneck (32 ms per step against 814 ms for the policy). The vision encoder takes 230 ms in the released bfloat16 weights but 39 ms in float16: Turing GPUs have no bfloat16 tensor cores and emulate it. The action expert accounts for 18% of the FLOPs of a re-plan but 63% of its latency (10×51 ms), and its step time is the same in both precisions: a pass over 50 action tokens is only 11.5 GFLOP but launches several hundred kernels, so at batch one the GPU waits on launches rather than arithmetic.

Successful and failed trajectories. Successes finish in a median of 100 steps; all 46 failures run to the 280-step limit after at least one grasp. From the executed gripper commands (object poses are not logged), 28 failures release the bowl off target after one or two grasps, 9 still hold it at the limit, and 9 grasp three or more times; task 3 (bowl on the cookie box) fails in 10 of 20 episodes. Fresh flow noise at every re-plan makes consecutive chunks differ about as much as independent samples: on 581 calibration steps of LIBERO-Object, two 10-step chunks for the same observation with different noise differ by 1.37 (L2 norm of the first normalised action), and the shifted previous chunk differs from the current one by 1.31. The gripper command chatters accordingly: failed episodes contain 12.6 gripper segments of at most 3 steps per 100 steps, successful ones 2.7.

Strengths. **S1 Small footprint:** 1.16 GB of weights and 21 MB of activations per re-plan (float16) fit a 16 GB T4 with room for several processes. **S2 Prefix caching:** the expert reuses the cached prefix, so a flow step costs 52 ms against 304 ms for the prefix. **S3 Few-step flow:** with $S{=}3$ the first action deviates from $S{=}10$ by 0.32, a quarter of the sampling spread (1.37), and SR does not drop (Sec. 5).

Limitations. **L1 Precision mismatch:** bfloat16 weights run emulated on Turing. **L2 Launch-bound expert:** the flow loop dominates latency at batch one while using little arithmetic. **L3 Need for fresh images:** executing 5 actions per chunk, or re-planning with fresh proprioception but a 5-step-old image, loses 12.0–12.5 points (Sec. 5), so an episode needs a full re-plan at every step. **L4 Temporally inconsistent actions:** independent noise at every re-plan produces gripper chatter, concentrated in failures.

5. Improvement: Cheaper, not Less, Re-planning

Bottleneck and hypotheses. A successful episode (100 steps) spends about 79 s in the policy and 3 s in the simulator. **H1 (re-plan less):** computation can be shared across steps without lowering SR, by executing k actions per chunk (open loop), by also re-planning when the next queued action switches the gripper, or by re-planning with fresh proprioception on cached image and language keys and values refreshed every 5 steps (stale image). **H2 (re-plan more cheaply):** each re-plan can be made cheaper while computing the same function up to precision and discretisation: float16 removes the emulated arithmetic (same weights, projections kept in float32); $S{=}3$ removes discretisation slack (S3 in Sec. 4); and a CUDA graph removes the launches of the expert loop. For the graph, the prefix runs eagerly and the S-step Euler loop is captured once per S (0.25–2.2 s) with static buffers for noise, prefix cache and time steps, then replayed; its actions equal the eager ones exactly in all 8 settings. H2 predicts SR within the margin and a larger graph speed-up once precision is fixed, since launch overhead hides arithmetic.

Protocol. Differences in SR are paired (exact McNemar test, Newcombe 95% CI [14], Holm adjustment [5] over comparisons with the released policy); a variant is non-inferior if the lower bound exceeds -5 points, a margin fixed in advance. Latency is measured on isolated calls, because the closed-loop runs share each GPU between two processes.

Re-planning less lowers success (H1 rejected). Calling the vision encoder about five times less often costs 12.0–12.5 points of SR (Tab. 4); with $k{=}10$ the drop of 9.5 points is not significant after adjustment. The event trigger removes most gripper chatter (1.2 short segments per 100 steps against 5.0) without recovering success, and fresh proprioception does not substitute for a current image. A SmolVLA trained from scratch on LIBERO benefits from longer execution horizons [16]; this checkpoint, fine-tuned with $n_{\text{exec}}{=}1$, does not.

	bfloat16 (released)		float16	
Euler steps S	eager	graph	eager	graph
10 (released)	789	572	616	178
3	433	372	245	**112**
2	386	345	191	103
1	335	318	136	93
Fit $a{+}bS$	$284{+}50.5S$	$289{+}28.3S$	$84{+}53.3S$	$84{+}9.4S$

Table 3. Latency of one re-plan (ms, median of 60 calls, T4, batch 1); least-squares fit over S (residuals $\leq$2.7 ms).

Variant	Vis./100	SR	ΔSR [95% CI]	p_{Holm}
Released (bf16, $S{=}10$)	100	77.0	–	–
H1: re-plan less				
Open loop, $k{=}5$	20.1	64.5	-12.5 [-20.3, -4.5]	0.012
Open loop, $k{=}10$	10.2	67.5	-9.5 [-17.3, -1.6]	0.074
$k{=}5$ + gripper events	21.5	65.0	-12.0 [-18.7, -5.2]	0.006
Stale image ($k{=}5$, $S{=}3$)	24.6	65.0	-12.0 [-19.1, -4.7]	0.009
H2: re-plan more cheaply				
bf16, $S{=}3$	100	79.0	$+2.0$ [-3.8, $+7.8$]	1.000
fp16, $S{=}3$	100	77.0	0.0 [-5.8, $+5.8$]	1.000
fp16, $S{=}2$	100	[–]	[–]	[–]
fp16, $S{=}1$	100	[–]	[–]	[–]

Table 4. Closed-loop results on 200 paired LIBERO-Spatial episodes. Vis./100: vision-encoder passes per 100 control steps.

Re-planning more cheaply preserves success (H2). Along released $\rightarrow$ float16 $\rightarrow$ $S{=}3$ $\rightarrow$ graph, a re-plan takes 789, 616, 245 and 112 ms, factors of 1.28, 2.51 and 2.19 ($7.05\times$ in total; Tab. 3). The fits explain these factors. Without the graph, an Euler step costs 50.5 ms in bfloat16 and 53.3 ms in float16, independent of the arithmetic: the loop is launch-bound. With the graph, it costs 28.3 and 9.4 ms: removing launches exposes the emulated arithmetic, so the graph gains $1.38\times$ in bfloat16 but $3.46\times$ in float16 at $S{=}10$. On 200 pairs, neither $S{=}3$ variant differs significantly from the released 77.0%, but for float16 the lower bound (-5.8) does not yet clear the margin. **[500 pairs: non-inferiority. S-curve. Fixed noise vs fp16 $S{=}3$ (SR, chatter). In-loop latency and graph equivalence.]**

Trade-offs and next bottleneck. The float16 gain is specific to GPUs without bfloat16 tensor cores; elsewhere only the step and graph gains would apply (not measured). Float16 changes actions by 0.010 on average and 0.049 at most (mean magnitude 0.17), with no overflow observed; the graph needs static shapes and one capture per S. At 112 ms per re-plan plus 32 ms of simulation the loop runs at about 7 Hz, below the 20 Hz control rate. By the fit

$T(S) \approx 84 + 9.4S$ ms, the vision-language prefix takes 75% of a re-plan at $S{=}3$ and even $S{=}1$ would need 93 ms; further gains require a cheaper prefix, which changes the model and needs training.

6. Conclusion

For the released SmolVLA checkpoint on a T4, the cost of a re-plan is set less by FLOPs than by emulated bfloat16 and a launch-bound flow loop. Sharing computation across steps lowers success by about 12 points, whereas removing these overheads and the discretisation slack makes a re-plan 7.0× cheaper without a measurable loss on 200 paired episodes. **[Update with the 500-pair decision and the S-curve.]** **Limitations:** one checkpoint, one LIBERO suite, one GPU type and simulation only; the baseline (77.0%) is below the 90% of the paper configuration. **Future work:** the remaining 84 ms lie in the vision-language prefix, which calls for distillation into a smaller backbone, and embedded GPUs or exported runtimes should be evaluated with the same paired closed-loop test [6].

References

[1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1

[2] Kevin Black, Manuel Y Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025. 1

[3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023. 1

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020. 1

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. 3

[6] Rafiqul Islam. When faster vla deployment changes closed-loop behavior: Task success-latency analysis of smolvla across pytorch and onnx variants. *arXiv preprint arXiv:2609.14146*, 2026. 1, 2, 4

[7] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 1

[8] LeRobot contributors. Cannot reproduce smolvla results on libero benchmark. GitHub issue huggingface/lerobot #2354, https://github.com/huggingface/lerobot/issues/2354, . Accessed October 2026. 2

[9] LeRobot contributors. mujoco>=3.4.0 silently breaks the initial-state premise of libero_spatial task 5. GitHub issue huggingface/lerobot #4390, https://github.com/huggingface/lerobot/issues/4390, . Accessed October 2026. 2

[10] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024. 1

[11] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 2

[12] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, pages 44776–44791, 2023. 1

[13] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running vlas at real-time speed. *arXiv preprint arXiv:2510.26742*, 2025. 1

[14] Robert G Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998. 3

[15] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025. 1

[16] Kohei Sendai et al. Leave no observation behind: Real-time correction for vla action chunks. *arXiv preprint arXiv:2509.23224*, 2025. 1, 3

[17] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 1, 2

[18] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. 2

[19] Zhipeng Tang, Xinda Chen, Weining Rao, Xiao Li, Wenting Tan, Yuning Wang, Xiao Shi, and Xiaofang Zhao. Fewer steps, better actions: Rethinking flow-matching inference for vla policies. *arXiv preprint arXiv:2609.21216*, 2026. 1, 2

[20] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025. 1

[21] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. Vla-cache: Towards efficient vision-language-action model via adaptive token caching in robotic manipulation. *arXiv preprint arXiv:2502.02175*, 2025. 1

[22] Yantai Yang et al. Efficientvla: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*, 2025. 1

[23] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. In *Advances in Neural Information Processing Systems*, 2024. 1